

ISG 算法插件开发规范 v1.0

概述

ISG 算法插件是一个标准 Python 模块（`.py` 文件），实现 `run(payload, context)` 入口函数，并声明模块级 `PARAMETERS` 列表。插件注册后，系统自动反射参数列表，在 UI 中生成对应的参数配置控件。

文件结构

代码块

```
1  # -*- coding: utf-8 -*-
2  """插件简要说明。"""
3  from __future__ import annotations
4
5  from pathlib import Path
6  from typing import Any
7
8  # =====
9  # PARAMETERS — 参数声明（必填）
10 # =====
11
12 PARAMETERS: List[dict[str, Any]] = [
13     {
14         "name": "threshold",          # 参数变量名（英文，代码中使用此 key 读取）
15         "type": "float",              # 类型: string / int / float / bool / select
16         "label": "阈值",              # UI 中显示的中文名称
17         "default": 0.5,              # 默认值
18         "min": 0.0,                  # 最小值（仅 int/float 类型生效，可选）
19         "max": 1.0,                  # 最大值（仅 int/float 类型生效，可选）
20         "options": [],               # 枚举选项列表（仅 select 类型生效）
21         "description": "判定阈值",   # 参数说明文本
22         "required": False,           # 是否必填
23     },
24 ]
```

```

25
26 # =====
27 # run - 算法入口 (必填)
28 # =====
29
30 def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
31     """算法执行入口。
32
33     Args:
34         payload: 包含 parameters, input, output 等字段 (见下文)
35         context: 提供进度上报、日志、取消检查等方法
36
37     Returns:
38         {"ok": True, "outputs": [...], "logs": [...]} 成功
39         {"ok": False, "error_code": "...", "message": "..."} 失败
40     """
41     ...

```

PARAMETERS 字段规范

字段	类型	必填	说明
<code>name</code>	str	是	参数变量名，英文小写+下划线，如 <code>threshold</code> 。代码中通过 <code>parameters.get("threshold")</code> 获取
<code>type</code>	str	是	<code>string</code> / <code>int</code> / <code>float</code> / <code>bool</code> / <code>select</code>
<code>label</code>	str	是	UI 显示名称，中文，如 <code>"检测阈值"</code>
<code>default</code>	*	是	默认值，类型须与 <code>type</code> 匹配
<code>min</code>	float	否	最小值，仅 <code>int</code> / <code>float</code> 生效

max	float	否	最大值，仅int / float 生效
options	list	否	枚举选项，仅 select 生效，如 ["A", "B", "C"]
description	str	否	参数说明文本
required	bool	否	是否必填，默认 false

类型约定

type 值	default 示例	说明
string	"normal"	字符串
int	100	整数
float	0.5	浮点数
bool	True / False	布尔值
select	"A"	枚举选择，options 必填且非空

> 注意： 不再使用 `number`、`integer`、`json` 等类型名。整数用 `int`，浮点数用 `float`。

run() 函数规范

函数签名

代码块

```
1 def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
```

payload 结构

代码块

```
1 payload = {
2     "algorithm_key": "generation.image.geometric_transform", # 算法唯一标识
3     "parameters": { # 用户配置的参数字典, key = PARAMETERS 中的 name
4         "rotation_degrees": 10.0,
5         "scale": 1.0,
6     },
7     "input": {
8         "dataset_id": 1, # 源数据集 ID
9         "dataset_path": "/data/datasets/abc", # 源数据集路径
10        "samples": [ # 样本列表
11            {
12                "id": 1,
13                "sample_path": "/data/datasets/abc/img_001.jpg",
14                "modality": "image",
15                "labels_json": ["ship"],
16            },
17        ],
18    },
19    "output": {
20        "output_dir": "/data/tasks/42/output", # 输出目录 (已创建)
21    },
22    "target_count": 100, # 期望生成数量 (生成/增强类)
23 }
```

context 方法

方法	说明
<code>context.set_progress(percent, message)</code>	更新进度 0-100, message 为描述文本

<code>context.log(level, message, payload)</code>	记录日志, level: "info" / "warn" / "error"
<code>context.is_cancel_request() -> bool</code>	检查是否收到取消请求, 应周期性检查

返回值

成功（清洗建议类）：

代码块

```
1  {
2      "ok": True,
3      "suggestions": [
4          {
5              "sample_id": 1,
6              "issue_type": "blur",
7              "suggested_action": "repair",
8              "confidence": 0.85,
9              "message": "图像模糊度 45.2",
10             "details": {"blur_score": 45.2, "output_path":
11                 "/path/to/repaiored.jpg"},
12         },
13     ],
14     "logs": [],
15 }
```

成功（生成/增强类）：

代码块

```
1  {
2      "ok": True,
3      "outputs": [
4          {
5              "source_sample_id": 1,
6              "output_path": "/data/tasks/42/output/img_001_geo_0000.jpg",
7              "relative_path": "img_001_geo_0000.jpg",
8              "metadata": {"method": "geometric_transform", "parameters": {...}},
9              "status": "created",
10          }
11     ]
12 }
```

```
10     }
11     ],
12     "logs": [],
13 }
```

失败:

代码块

```
1  {
2      "ok": False,
3      "error_code": "NO_INPUT_SAMPLES", # 大写+下划线
4      "message": "无输入样本",
5  }
```

取消:

代码块

```
1  {
2      "ok": False,
3      "error_code": "CANCELLED",
4      "message": "任务已取消",
5  }
```

读取参数的推荐方式

代码块

```
1  def run(payload: dict[str, Any], context: Any) -> dict[str, Any]:
2      parameters = payload.get("parameters", {}) or {}
3
4      # 数值类型
5      threshold = float(parameters.get("threshold", 0.5))
6      max_iter = int(parameters.get("max_iterations", 100))
7
8      # 字符串/选择类型
9      mode = str(parameters.get("mode", "normal"))
```

```

10
11     # 布尔类型 - 使用辅助函数
12     enable = _as_bool(parameters.get("enable_logging", True))
13
14     # 列表类型
15     stop_words = list(parameters.get("stop_words") or [])
16
17     ...
18
19     def _as_bool(value) -> bool:
20         """将任意值转为布尔类型。"""
21         if isinstance(value, bool):
22             return value
23         return str(value).strip().lower() in {"1", "true", "yes", "y", "是"}

```

命名约定

1. **参数 name 字段**：英文小写 + 下划线，如 `blur\_threshold`、`learning\_rate`
2. **参数 label 字段**：简短中文，如 `“模糊阈值”`、`“学习率”`
3. **算法 key**：用 `.` 分隔命名空间，如 `generation.image.geometric\_transform`
4. **算法 name**：UI 展示用中文名，如 `“几何变换”`

完整示例

参见 `plugins/user/_TEMPLATE.py`。

注意事项

1. `PARAMETERS` 必须是模块级变量，不能定义在函数内部
2. 无参数插件仍需声明 `PARAMETERS = []`（空列表）
3. 不要在 `run()` 中修改 `PARAMETERS`
4. 所有文件 I/O 使用 `pathlib.Path`，路径用 `/` 分隔
5. 大文件或耗时操作应周期性调用 `context.is_cancel_requested()`

6. 插件可放置在 `plugins/user/` 目录，或通过 `module_path` 引用已安装的包